Tableau Resetmaxxing for Fun and Profit

(a crazy idea that works surprisingly well)

Introduction and Novel Idea

eliminatingModels introduce SATCoP [**eliminatingModels**], a tableau prover based on LeanCoP [**Otten2023leanCoP20**] that runs a separate SAT solver along fed with grounded atoms from the tableau prover. If the SAT solver reaches an unsatisfiable state, then we can prove that the entire tree is unsatisfiable.

However, other than the underlying SAT solver, the tableau prover in the implementation of SATCoP only has a few improvements over the original LeanCoP.

1. Randomisation of literals at `Start` and `Extension` clauses.
2. Deepening control: only increase maximum height when the SAT model is stale.
3. And a big improvements present in **9.1 A Note from the Future** and in CASC: SAT-guided preference: reorder extensions by the SAT model's opinions of the literals.

I improved the current model of `SATSolve` with the reordering in **??** to generate `SATSolveLiteral`. This already provides a big improvement over the baseline.

For the most part, this guidance tries to maximise the chance of closing the tableau rather than trying to create UNSAT on the underlying SAT solver.

While experimenting with choices for SAT-based guidance[**apr22˙pres**], I found some big improvements in specifically changing the tableau search to achieve this. In particular, **resetting the tableau search early and often to generate different statements for the SAT solver considerably improves the overall model**.

This makes some sense intuitively: instead of trying to close the tableau, we are trying to generate the maximum amount of valid SAT statements to "close" the SAT prover.

As far as I can tell this is sound (although very incomplete).

Implementation

We compare these strategies for the SAT prover along with two control strategies: `FirstAction` and `LeanCoPCut`.

1. `SATSolve` Plain SATCoP[**eliminatingModels**]. Last week's best model.
2. `SATSolveLiteral` SATCoP with the extension reordering.
3. `SATResetAtStaleAction` Reset when no new actions to the SAT solver.
4. `SATResetAtStaleBacktrack` Reset when backgracking if no new actions have been added to the SAT solver.
5. `SATResetBandit` Tiny contextual bandit choosing between `Backtrack(1)`, `Backtrack(2)`, and `Reset(n + 1)` depending on whether the shadow CNF is still growing.

Overall Results

The CRS3 tier is currently hammered, so most of these tests were done locally. They should be roughly accurate anyway.

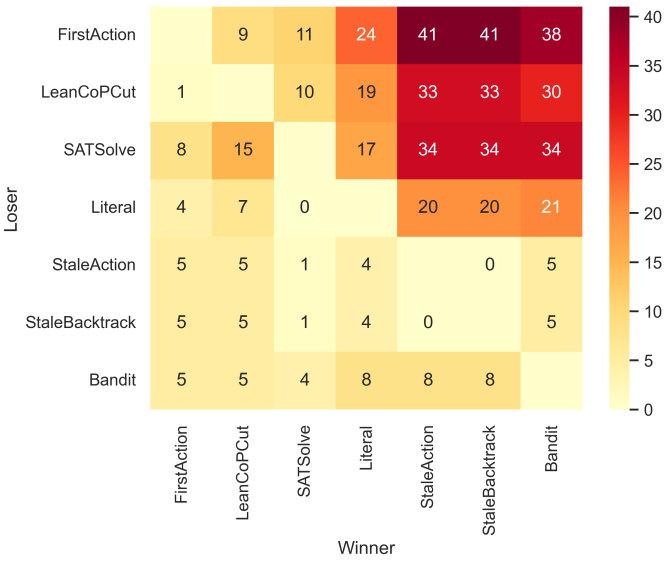


Figure 1: Winners and losers of different SAT policies in a subset of 501 problems [**chvalovsky2021characteristic**].

Some Interesting Results

Problem	FirstAction		LeanCoPCut		SATSolve		SATSolveLiteral		SATResetAtStaleAction		SATResetAtStaleBacktrack		SATResetBandit	
	ms	steps	ms	steps	ms	steps	ms	steps	ms	steps	ms	steps	ms	steps
NLP004-1.p	—	—	—	—	60,007	15,364	—	—	4,986	3,505	5,025	3,505	1,077	884
KRS072+1.p	—	—	—	—	60,003	15,317	—	—	3,064	1,605	3,798	1,605	8,505	3,028
FLD055-1.p	—	—	—	—	60,003	7,986	—	—	4,797	1,884	4,962	1,884	9,856	2,172
GEO262+3.p	—	—	—	—	60,003	16,831	—	—	6,528	1,802	5,667	1,802	7,592	2,696
GEO233+1.p	—	—	—	—	60,002	15,962	—	—	13,662	3,219	13,566	3,219	6,892	1,870
SYN209-1.p	60,001	42,135	60,000	42,642	60,007	10,886	55,014	26,905	1,396	481	1,409	481	3,777	1,260
NUM411+1.p	60,002	33,327	44,678	26,538	60,004	14,177	—	—	5,978	2,850	5,603	2,850	1,061	659
SYN154-1.p	60,002	43,785	60,002	42,510	38,491	9,279	37,034	18,150	270	95	244	95	1,649	550
SET677+3.p	60,061	3,638	60,082	3,751	60,010	8,754	32,935	11,136	1,284	340	826	340	2,416	366
GEO202+3.p	—	—	26,074	13,927	60,000	13,139	—	—	6,782	2,222	7,126	2,222	9,544	2,399

Table 1: Ten problems on which at least one reset-based policy clearly beats all non-reset baselines. Unsolved entries are shown as — in both time and step columns.